COMPUTER NETWORKS - UE24CS252B

Lab 2: Understanding Persistent and Non-persistent HTTP Connections

Objective: To understand persistent and non-persistent HTTP connections and corresponding performance impact.

Task:

Create a web page with N (e.g. 10) embedded images. Each image should be of minimum 2 MB size. Configure your browser (Firefox) with following settings (each setting requires repeat of experiment):

a. Non persistent connection
b. 2 persistent connections
c. 4 persistent connections

I. System Requirements

Software

- Ubuntu Linux (Server & Client machines / VMs)
- Apache HTTP Server
- Mozilla Firefox
- Wireshark
- VirtualBox / VMware (if using VMs)

Hardware / Network

- Two systems (or two VMs): **Server** and **Client**
- Connected via a **bridged network** (if using 2 systems)
- Or, Create groups of 2 such that one student's desktop will be the server, the other will be the client.

II. Theory: Apache Server, Client, and Host

Apache HTTP Server

The Apache HTTP server is the most widely-used web server in the world. It provides many powerful features including dynamically loadable modules, robust media support, and extensive integration with other popular software.

Client and Host Interaction

In a web system, the client (e.g., a browser) sends HTTP requests to the Apache server, and the server (host) listens for those requests and responds with the requested resources. Apache acts as the intermediary between the client and the files or services hosted on the server machine, handling everything from static page requests to dynamic content generation.

Persistent vs. Non-Persistent Connections

When HTTP clients and servers communicate, they do over a TCP connection:

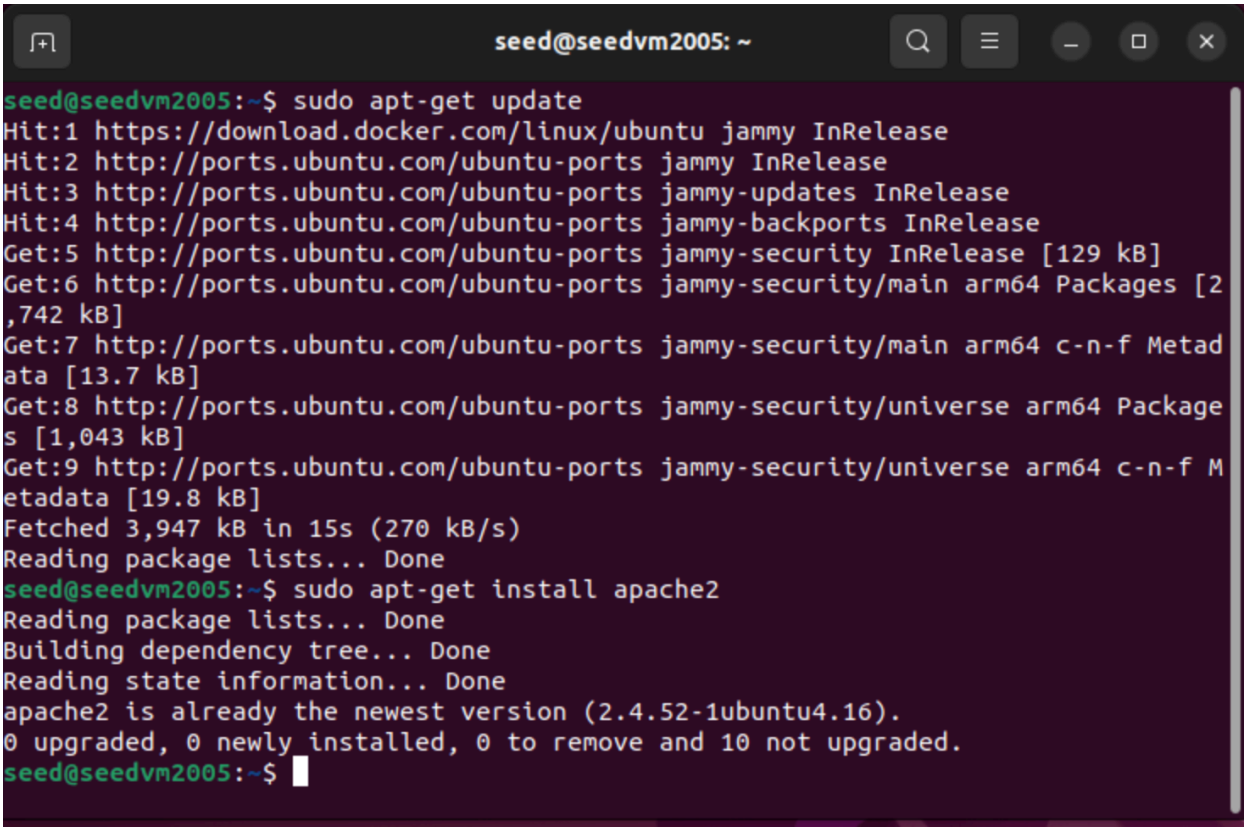
- Non-persistent connections open a new TCP connection for each object requested, increasing overhead.
- Persistent connections (also called HTTP Keep-Alive) allow multiple HTTP requests/responses over a single TCP connection, reducing latency and resource usage.

Apache supports persistent HTTP connections via its KeepAlive directive (KeepAlive On/Off), which enables clients and the server to reuse the same TCP connection for multiple HTTP requests as long as both sides agree.

III. Apache Server Setup (Server Machine)

Step 1: Install Apache

```
sudo apt-get update
sudo apt-get install apache2
```

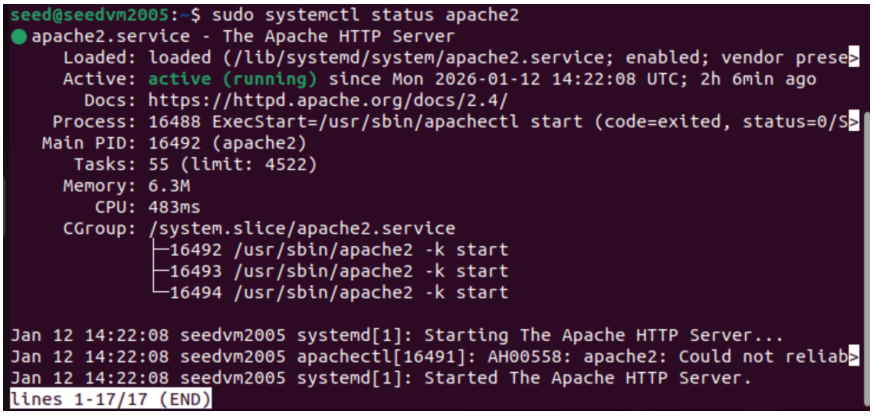


Step 2: Verify Apache Status

```
sudo systemctl status apache2
```

If not running:

```
sudo systemctl restart apache2
```



Step 3: Verify Web Server

Open a browser on the **client** and enter:

`http://<server-ip>`

Check server ip through ifconfig (for linux/MacOS) or ipconfig (for windows) command.

Server side:

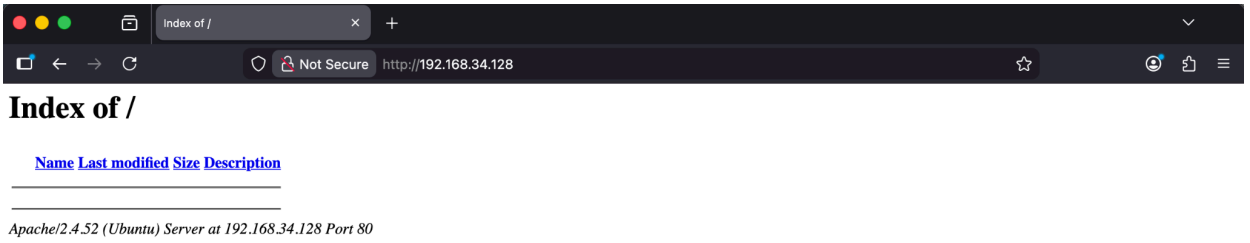
```
seed@seedvm2005:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 3a:8d:cc:23:95:e4  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

ens160: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.34.128  netmask 255.255.255.0  broadcast 192.168.34.255
        inet6 fe80::20c:29ff:fed3:f05b  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:d3:f0:5b  txqueuelen 1000  (Ethernet)
        RX packets 163331  bytes 194866063 (194.8 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 97963  bytes 114138592 (114.1 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 46  memory 0x3fe00000-3fe20000

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 42558  bytes 108807052 (108.8 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 42558  bytes 108807052 (108.8 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Note: the apache default page must be accessible at 127.0.0.1 on the server side and corresponding ip address (in this case, port ens160 and ip = 192.168.34.128).

Client side:



The Apache default page or previously set index file should load.

IV. Web Page Preparation

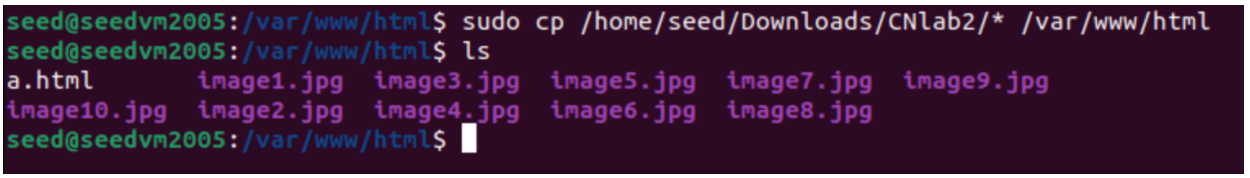
Step 4: Prepare Content

- Use 10 images, each of minimum size 2 MB
- Use the HTML file provided by the faculty

Step 5: Copy Files to Web Directory

```
sudo cp /path/to/images/* /var/www/html/
```

Note: path/to/images/* can be checked by pwd command



HTML file location:

```
/var/www/html/file_name.html
```

Ensure correct permissions:

```
sudo chmod -R 755 /var/www/html/
```

V. Browser Configuration (Client Machine)

Open Firefox and type:

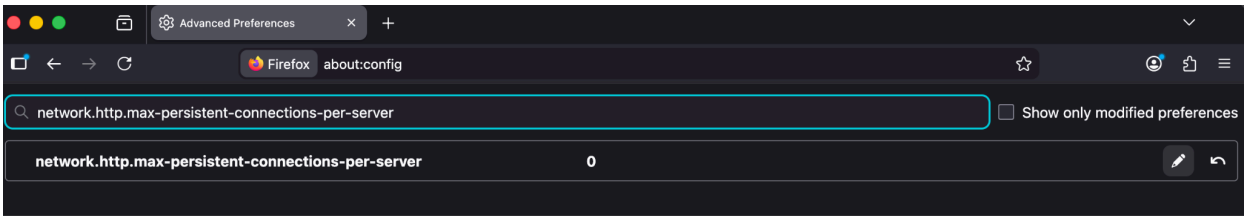
```
about:config
```

Search for the following settings and modify them according to the experiment:

Parameter	Non-Persistent	Persistent
network.http.keep-alive	false	true
network.http.max-persistent-connections-per-server	0	2 / 4 / 6 / 8 / 10

IMPORTANT: Clear browser cache **before each experiment**.





Note: these values have to be changed depending on the experiment and deliverables.

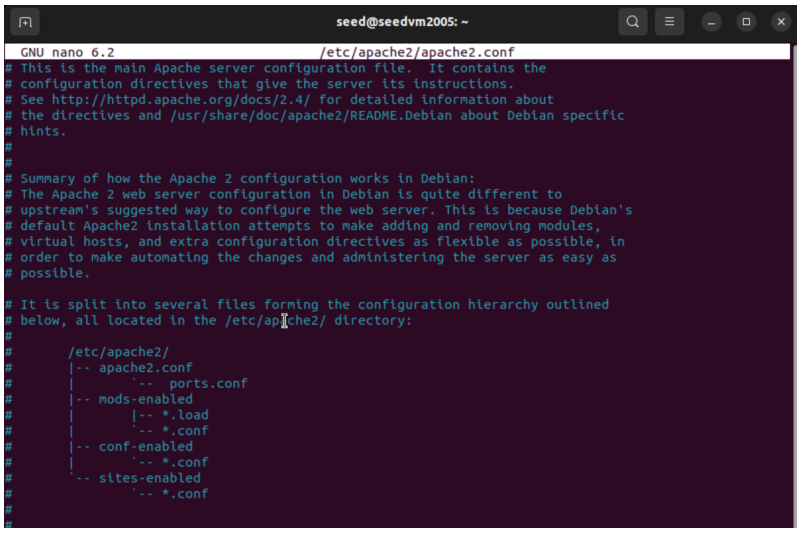
Experiment 1: Non-Persistent HTTP Connection

Server Configuration

Edit Apache configuration:

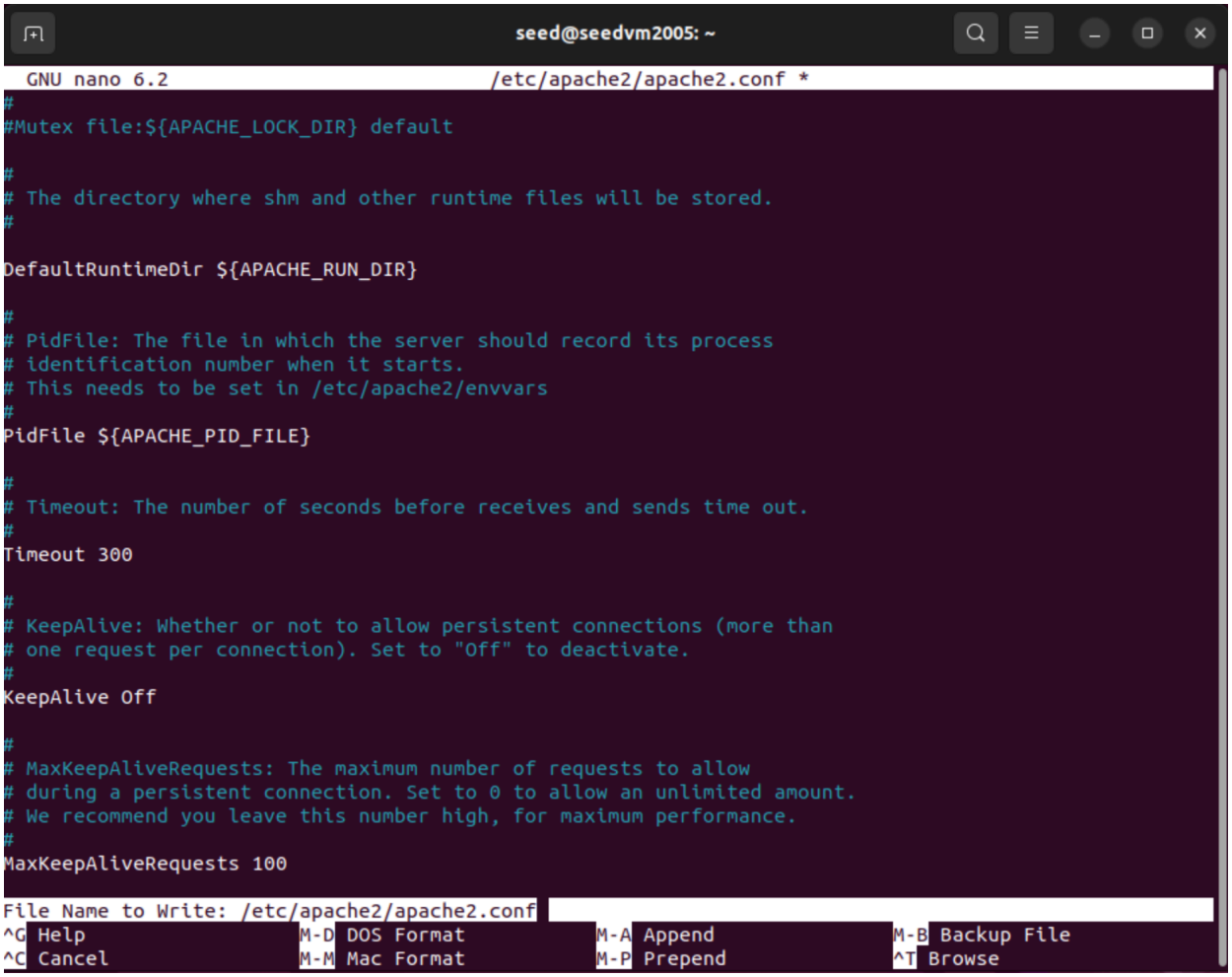
1. In the terminal, enter:

```
sudo nano /etc/apache2/apache2.conf
```



2. Set:

```
KeepAlive Off
```



Useful nano shortcuts:

Saving a file sequence: Ctrl + X -> Y -> enter

Search: Ctrl + W

3. Restart Apache:

```
sudo systemctl restart apache2
```

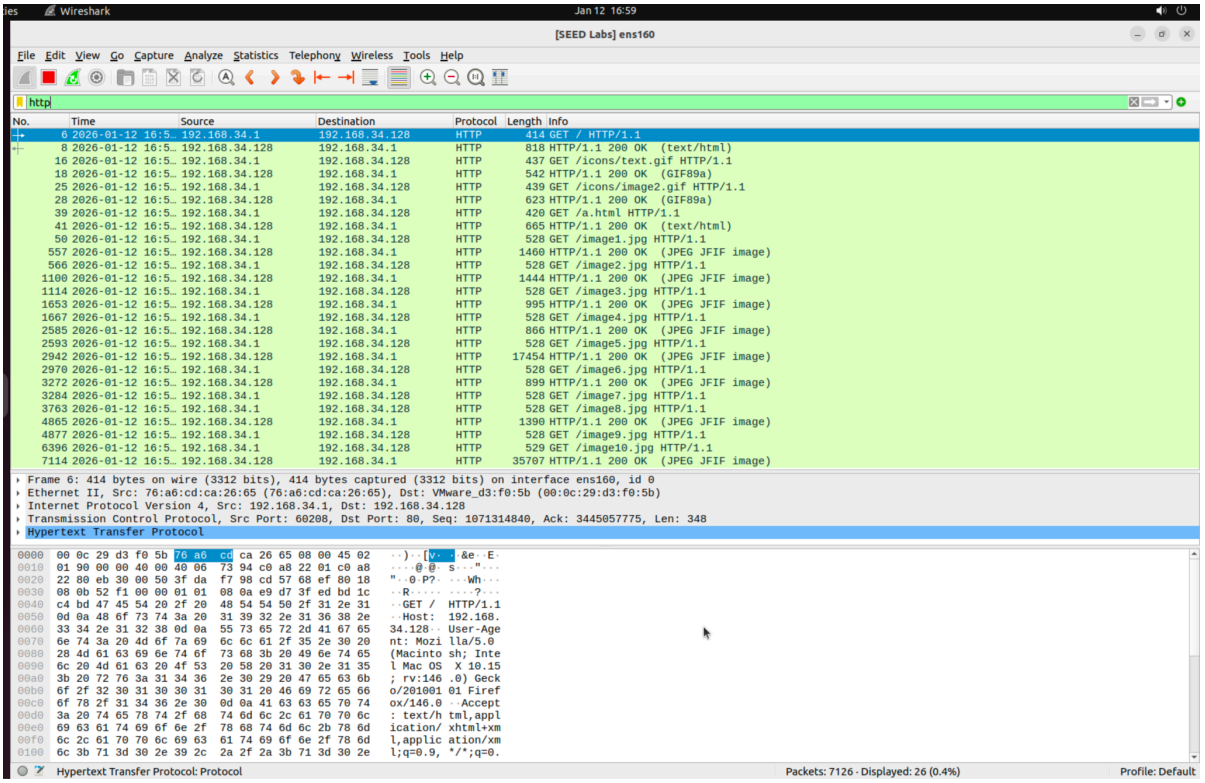
Client Configuration

Set max-persistent-connections-per-server = 0 in firefox settings, as directed above.

Capture Steps:

1. Start Wireshark on the **server.**
2. Choose **en0** or in this case, **ens160** (or any corresponding port acting as a bridge between VM and host system).
3. Apply filter: **http**
4. Access the page from client:

   http://<server-ip>/file_name.html



4. Record the **total time** taken to download all 10 images

Note 1: The wireshark should capture the packets between the client and the server while the file is accessed.

Note 2: The images in the HTML page should have all the permissions specified through the server for the proper access.

Experiment 2: Persistent HTTP Connections

Case A: 2 Persistent Connections

1. **Client:** Change the firefox settings again and set **max-persistent-connections-per-server = 2.**
2. **Server:** Change the apache2 configuration files (apache2.conf) by rewriting **KeepAlive On.**
3. Restart Apache and repeat the capture steps.

Case B: 4+ Persistent Connections

1. Repeat the above steps (Case A 1-3) with: **max-persistent-connections-per-server = 4.**
2. Clear browser cache between every capture, and change **max-persistent-connections-per-server** accordingly (6/8/10).

V. Deliverables

A. Experiment Observations:

Capture traffic until all 10 images are loaded and note the **start and end timestamps** from Wireshark.

For each experiment and sub-part, calculate:

Total Load Time = End Time – Start Time

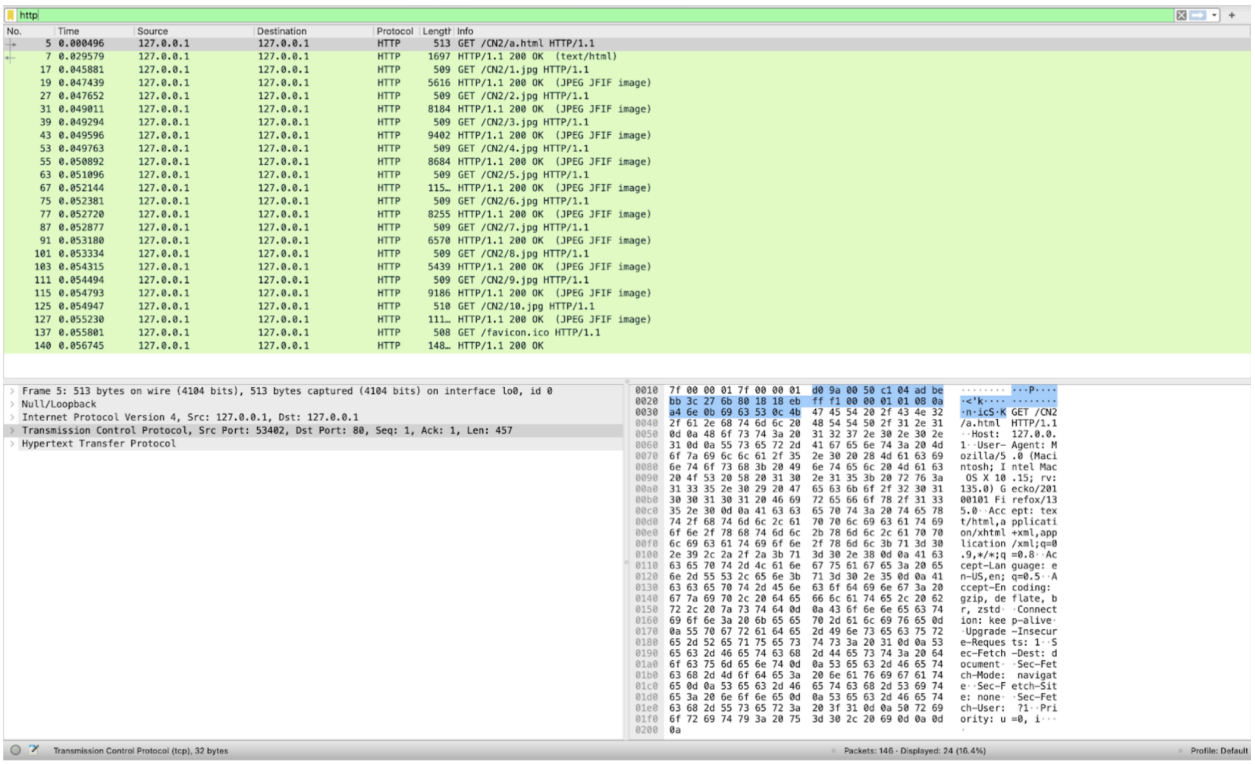
Record the values **directly from Wireshark timestamps**.

Observation table:

Connection Type	Persistent Connections	Start Time (s)	End Time (s)	Load Time (s)
Non-Persistent	0			
Persistent	2			
Persistent	4			

Persistent	6			
Persistent	8			
Persistent	10			

Example:



Start time = 0.00496s

End time = 0.056745s

Load time = End time - Start time = 0.051785s

B. Graph Plot:

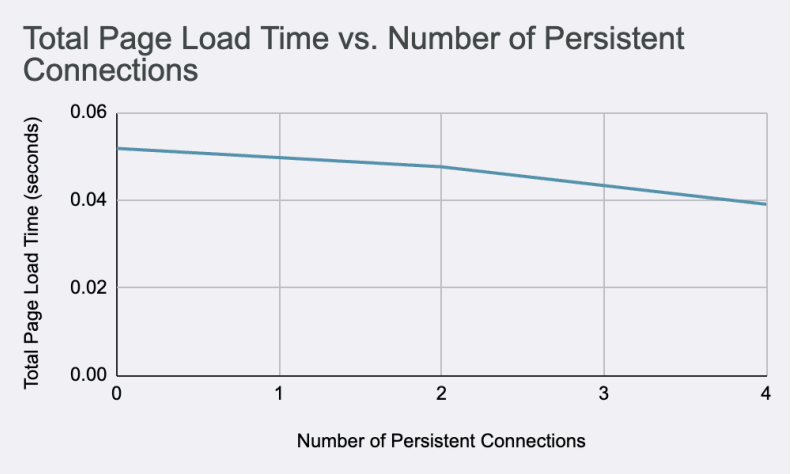
Students must plot a graph using the observed values.

Graph Requirements:

- X-axis: Number of Persistent Connections (0, 2, 4, 6, 8, 10)
- Y-axis: Total Page Load Time (seconds)

Students may use **Excel / LibreOffice Calc / Python / MATLAB**.

Example plot:



C. Screenshots Required for Submission:

1. Wireshark capture for non-persistent connection (all 10 images visible)
2. Wireshark capture for persistent connection – 2 connections
3. Wireshark capture for persistent connection – 4, 6, 8, 10 connections
4. Graph screenshot showing load time vs number of connections

Conclude by Summarizing how HTTP connection persistence impacts page load time and network efficiency.